Listing 5.17 `SK_semantic_native_functions.py` (abridged)

```
prompt_template_config = sk.PromptTemplateConfig(
    template=sk_prompt,
    name="tldr",
    template_format="semantic-kernel",
    execution_settings=execution_settings,
)     #1

recommend_function = kernel.create_function_from_prompt(
    prompt_template_config=prompt_template_config,
    function_name="Recommend_Movies",
    plugin_name="Recommendation",
)     #2


async def run_recommendation():     #3
    recommendation = await kernel.invoke(
        recommend_function,
        sk.KernelArguments(),
    )
    print(recommendation)


# Use asyncio.run to execute the async function
asyncio.run(run_recommendation())
###Output
Based on the list provided, it seems the user is a fan of the Matrix
franchise. Since they have watched all four existing Matrix movies, I
would recommend a…
```

#1 Creates the prompt template config for the prompt
#2 Creates an inline semantic function from the prompt
#3 Executes the semantic function asynchronously

Run the code, and you should see an output like listing 5.17. One important aspect to note is that the native function is registered with the kernel, but the semantic function is not. This is important because function creation doesn't register a function.

For this example to work correctly, the native function must be registered with the kernel, which uses the `import_plugin` function call—the first line in listing 5.17. However, the semantic function itself isn't registered. An easy way to register the function is to make it a plugin and import it.

These simple exercises showcase ways to integrate plugins and skills into chat or agent interfaces. In the next section, we'll look at a complete example demonstrating adding a plugin representing a service or GPT interface to a chat function.